IPD440 - APRENDIZAJE DE MAQUINAS

Clase 20: Monte Carlo Methods

Prof. Nicolás Torres
nicolas.torresr@usm.cl
Ingeniería Civil Telemática
Departamento de Electrónica
Universidad Técnica Federico Santa María

El método Monte Carlo

El método Monte Carlo es una técnica estadística que se utiliza para encontrar una solución aproximada mediante muestreo.

La aproximación será mejor cuando el tamaño de la muestra sea grande.

Tareas en RL

En el aprendizaje por refuerzo realizamos dos tareas importantes y son:

- **Tarea de predicción:** se proporciona una política π como entrada y tratamos de predecir la función de valor o función Q utilizando la política dada. Es decir, necesitamos determinar si la política dada es buena o mala.
- **Tarea de control:** no se proporciona ninguna política como entrada, el objetivo es encontrar la política óptima.

Monte Carlo prediction

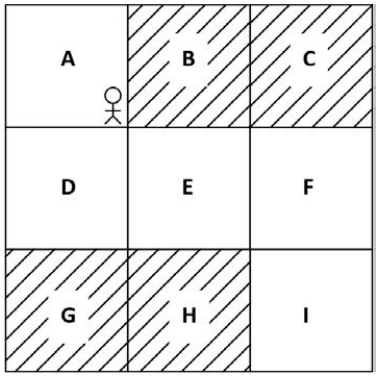
Para aproximar el valor del estado usando el método de Monte Carlo, tomamos muestras de episodios (trayectorias) siguiendo la política dada π durante algunas N veces y luego calculamos el valor del estado como el rendimiento promedio de un estado en todos los episodios muestreados.

$$V(s) \approx \frac{1}{N} \sum_{i=1}^{N} R_i(s)$$

Grid world environment

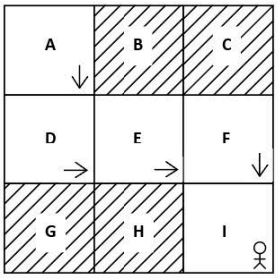
Nuestro objetivo es llegar al estado I desde el estado A sin visitar los estados sombreados, y el agente recibe una recompensa de +1 cuando visita los estados no sombreados y una

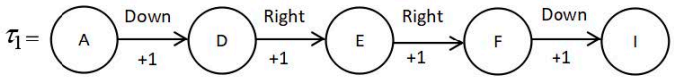
recompensa de -1 cuando visita los estados sombreados.



Episodio τ_1

Digamos que tenemos una política estocástica π. Supongamos que, en el estado A, nuestra política estocástica π selecciona acción hacia abajo el 80% del tiempo y acción derecha el 20% del tiempo, y selecciona acción derecha en los estados D y E y acción hacia abajo en los estados B y F el 100% del tiempo.

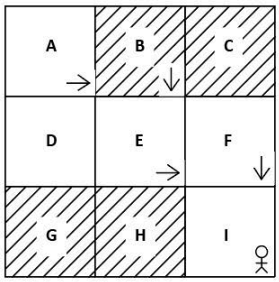


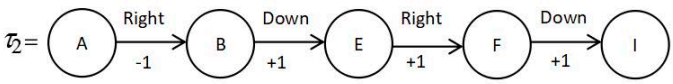


$$R_1(A) = 1 + 1 + 1 + 1 = 4$$

Episodio τ_2

Digamos que tenemos una política estocástica π. Supongamos que, en el estado A, nuestra política estocástica π selecciona acción hacia abajo el 80% del tiempo y acción derecha el 20% del tiempo, y selecciona acción derecha en los estados D y E y acción hacia abajo en los estados B y F el 100% del tiempo.

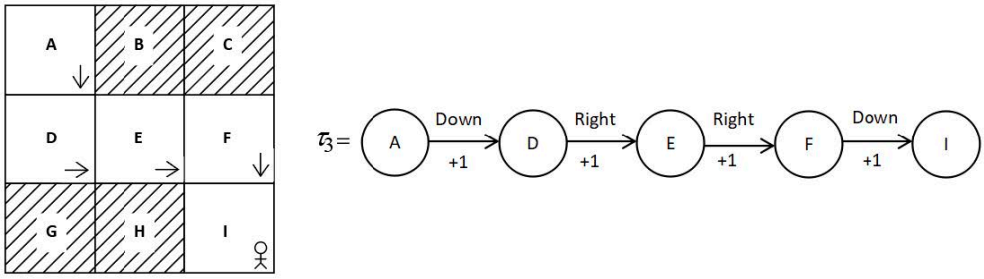




$$R_2(A) = -1 + 1 + 1 + 1 = 2$$

Episodio τ_3

Digamos que tenemos una política estocástica π. Supongamos que, en el estado A, nuestra política estocástica π selecciona acción hacia abajo el 80% del tiempo y acción derecha el 20% del tiempo, y selecciona acción derecha en los estados D y E y acción hacia abajo en los estados B y F el 100% del tiempo.



$$R_3(A) = 1 + 1 + 1 + 1 = 4$$

Calcular Valor Estado A

Por lo tanto, generamos tres episodios y calculamos el retorno del estado A en los tres episodios. Ahora bien, ¿cómo podemos calcular el valor del estado A? Si en el método de Monte Carlo, el valor de un estado se puede aproximar calculando el rendimiento promedio del estado en algunos episodios (trayectorias):

$$V(s) \approx \frac{1}{N} \sum_{i=1}^{N} R_i(s)$$

Necesitamos calcular el valor del estado A, por lo que podemos calcularlo simplemente tomando el rendimiento promedio del estado A en los 3 episodios como:

$$V(A) \approx \frac{1}{3} \sum_{i=1}^{3} R_i(A) = \frac{1}{3}(R_1(A) + R_2(A) + R_3(A)) = \frac{4 + 2 + 4}{3} = 3.3$$

MC prediction algorithm

Para entender bien el algoritmo de predicción de MC vamos a seguir un ejemplo paso a paso:

Paso 1

Inicialice los `total_return(s)` y `N(s)` para todos los estados en cero:

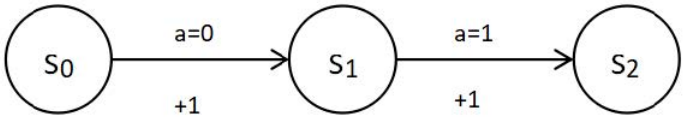
State	total_return(s)	N(S)
S_0	0	0
S_1	0	0

Paso 2

Digamos que tenemos una política estocástica π; en el estado S_0 nuestra política estocástica selecciona la acción 0 el 50% del tiempo y la acción 1 el 50% del tiempo, y selecciona la acción 1 en el estado S_1 el 100% del tiempo.

Iteración 1:

La etapa 2, genera un episodio usando la política de entrada dada π:



$$rewards = [1, 1]$$

Primero, calculamos el retorno del estado S_0 (suma de recompensas de S_0):

$$\begin{aligned} R(S_0) =& sum(rewards[0:]) \\ =& sum([1, 1]) \\ =& 2 \end{aligned} \tag{1}$$

Actualice el retorno total del estado S_0 en nuestra tabla como:

$$\begin{aligned} total_returns(S_0) =& total_returns(S_0) + R(S_0) \\ =& 0 + 2 \\ =& 2 \end{aligned} \tag{2}$$

Actualice la cantidad de veces que se visita el estado S_0 en nuestra tabla como:

$$\begin{aligned} N(S_0) =& N(S_0) + 1 \\ =& 0 + 1 \\ =& 1 \end{aligned} \tag{3}$$

Ahora, calculemos el retorno del estado S_1 (suma de recompensas de S_1):

$$\begin{aligned} R(S_1) =& sum(rewards[1:]) \\ =& sum([1]) \\ =& 1 \end{aligned} \tag{4}$$

Actualice el retorno total del estado S_1 en nuestra tabla como:

$$\begin{aligned} total_returns(S_1) =& total_returns(S_1) + R(S_1) \\ =& 0 + 1 \\ =& 1 \end{aligned} \tag{5}$$

Actualice la cantidad de veces que se visita el estado S_1 en nuestra tabla como:

$$\begin{aligned} N(S_1) =& N(S_1) + 1 \\ =& 0 + 1 \\ =& 1 \end{aligned} \tag{6}$$

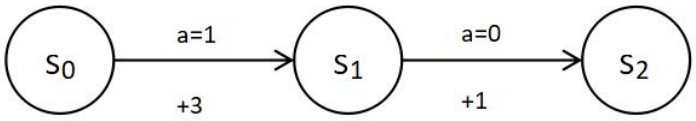
Tabla actualizada

Nuestra tabla actualizada, después de la iteración 1, es la siguiente

State	total_return(s)	N(S)
S_0	2	1
S_1	1	1

Iteración 2:

Digamos que generamos otro episodio usando la siguiente política π :



$$rewards = [3, 1]$$

Primero, calculamos el retorno del estado S_0 (suma de recompensas de S_0):

$$\begin{aligned} R(S_0) =& sum(rewards[0:]) \\ =& sum([3, 1]) \\ =& 4 \end{aligned} \tag{7}$$

Actualice el retorno total del estado S_0 en nuestra tabla como:

$$\begin{aligned} total_returns(S_0) =& total_returns(S_0) + R(S_0) \\ =& 2 + 4 \\ =& 6 \end{aligned} \tag{8}$$

Actualice la cantidad de veces que se visita el estado S_0 en nuestra tabla como:

$$\begin{aligned} N(S_0) =& N(S_0) + 1 \\ =& 1 + 1 \\ =& 2 \end{aligned} \tag{9}$$

Ahora, calculemos el retorno del estado S_1 (suma de recompensas de S_1):

$$\begin{aligned} R(S_1) =& sum(rewards[1:]) \\ =& sum([1]) \\ =& 1 \end{aligned} \tag{10}$$

Actualice el retorno total del estado S_1 en nuestra tabla como:

$$\begin{aligned} total_returns(S_1) =& total_returns(S_1) + R(S_1) \\ =& 1 + 1 \\ =& 2 \end{aligned} \tag{11}$$

Actualice la cantidad de veces que se visita el estado S_1 en nuestra tabla como:

$$\begin{aligned} N(S_1) &= N(S_1) + 1 \\ &= 1 + 1 \\ &= 2 \end{aligned} \tag{12}$$

Tabla actualizada

Nuestra tabla actualizada, después de la iteración 2, es la siguiente

State	total_return(s)	N(S)
S_0	6	2
S_1	2	2

Paso 3

Ahora podemos calcular el valor del estado como:

$$V(s) = \frac{\text{total_returns(s)}}{N(s)}$$

Por lo tanto,

$$V(S_0) = \frac{\text{total_returns}(S_0)}{N(S_0)} = \frac{6}{2} = 3$$

$$V(S_1) = \frac{\text{total_returns}(S_1)}{N(S_1)} = \frac{2}{2} = 1$$

Blackjack prediction with Monte Carlo

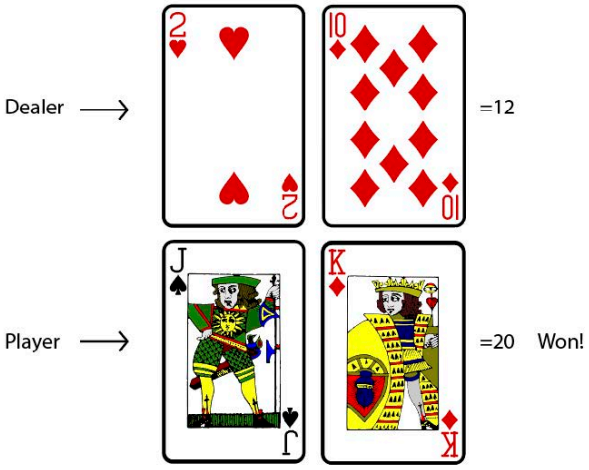
El Blackjack, también conocido como 21, es uno de los juegos de cartas más populares. El juego consta de un jugador y un crupier. El objetivo del jugador es que el valor de la suma de todas sus cartas sea 21 o un valor mayor que la suma de las cartas del crupier sin exceder 21. Si se cumple uno de estos criterios, el jugador gana el juego; de lo contrario, el crupier gana el juego.

Los valores de las cartas Jota (J), Rey (K) y Reina (Q) se considerarán 10. El valor del As (A) puede ser 1 u 11, según la elección del jugador, y el resto de cartas (2 a 10) su valor es nominal.

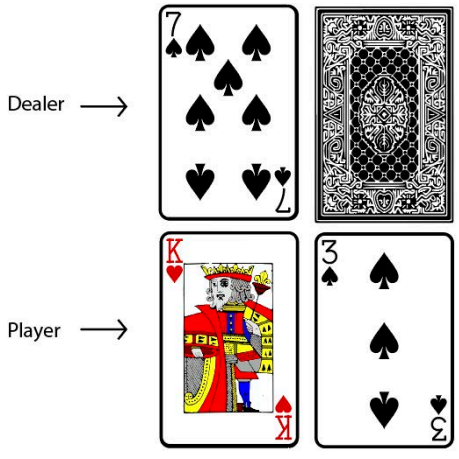
Caso 1



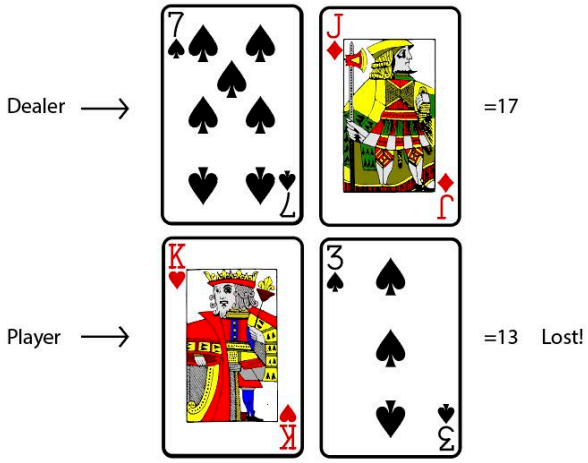
Caso 1: el jugador gana el juego.



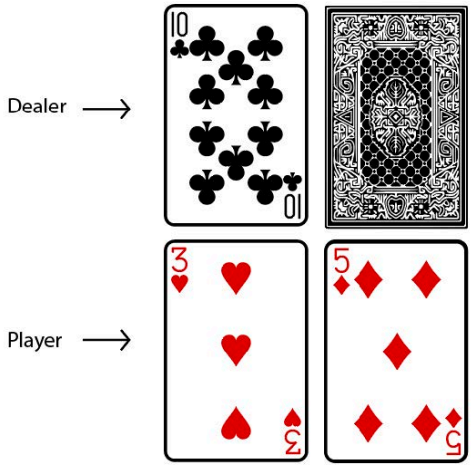
Caso 2



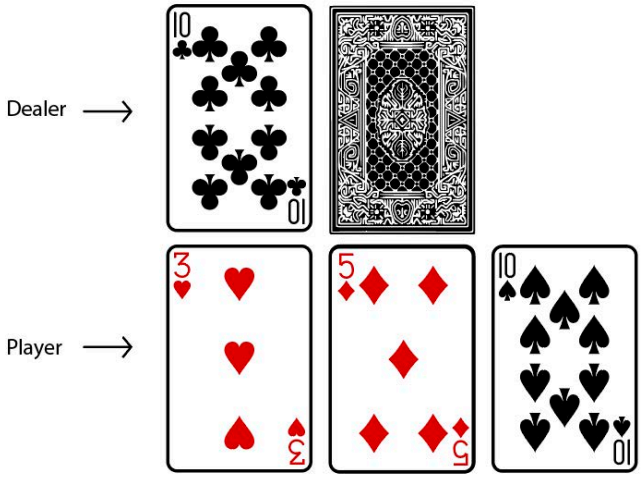
Caso 2: el jugador pierde el juego.



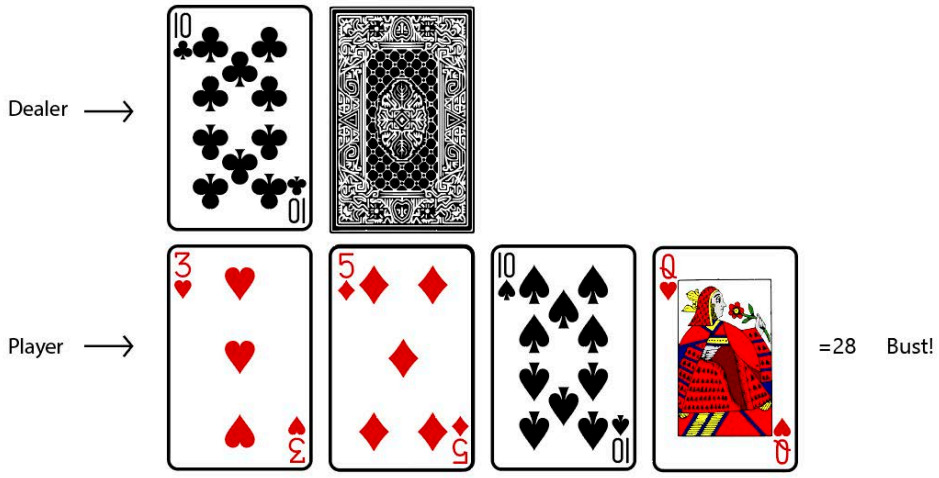
Caso 3



Caso 3: el jugador pide una nueva carta.



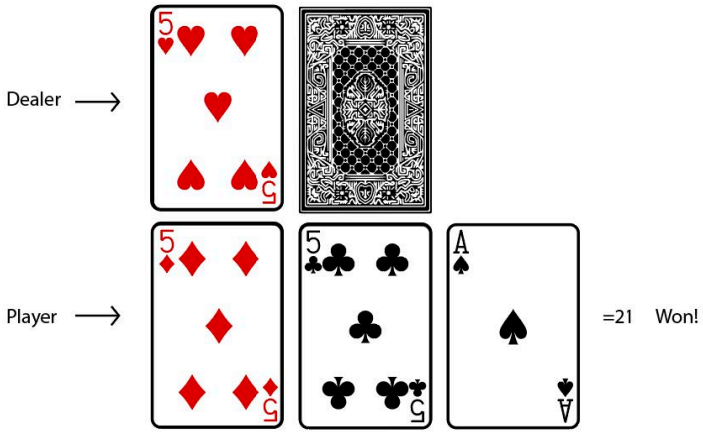
Caso 3: el jugador se pasa y pierde el juego.



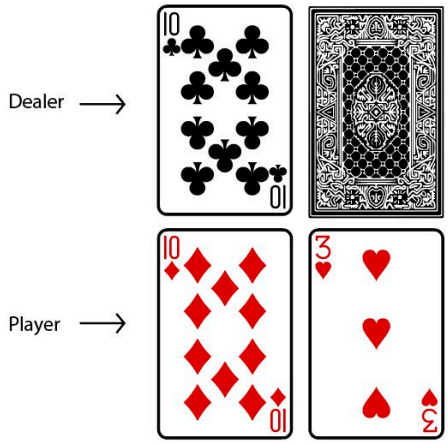
Caso 4



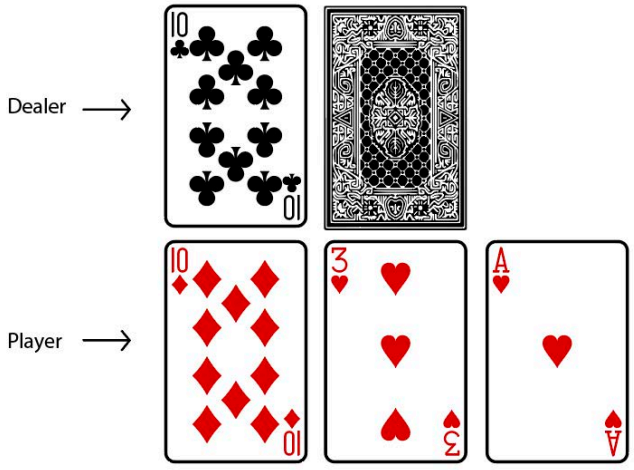
Caso 4: el jugador pide una nueva carta, sale un As y gana.



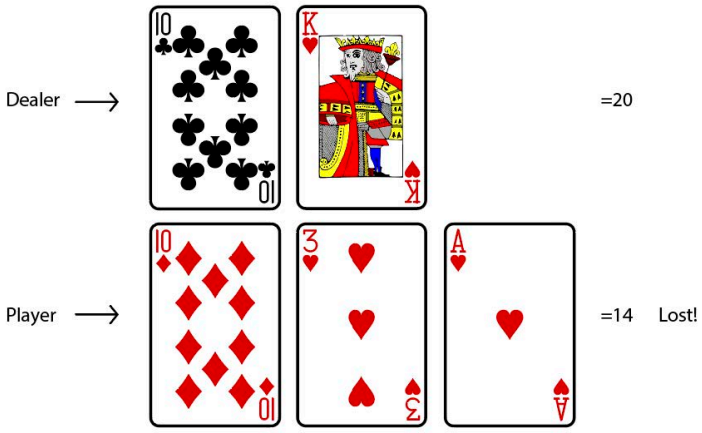
Caso 5



Caso 5: el jugador pide una nueva carta, sale un As y lo usa como 1.



Caso 5: el jugador no pide más cartas y pierde.



The blackjack environment in the Gym library

In [1]:

```
import gym
import pandas as pd
from collections import defaultdict
```

In [ ]:
```
env = gym.make('Blackjack-v1')
```

Estado

En el entorno del blackjack, el estado se representa como una tupla que consta de tres valores:

1. El valor de la suma de nuestras cartas.
2. El valor nominal de una de las cartas del crupier.
3. Valor booleano: Verdadero si tenemos un As utilizable y Falso si no tenemos un As utilizable.

In [4]:
```
state = env.reset()
print(state)
```

```
(5, 5, False)
```

Definiendo una política

Definimos la política de entrada cuya función de valor se predecirá en los próximos pasos.

Nuestra función de política recibe el estado como entrada y si la suma de nuestras cartas (índice `0` del estado) es mayor que 19, entonces devolverá la acción 0 (no pedir otra carta), de lo contrario devolverá la acción 1 (pedir otra carta).

In [3]:
```
def policy(state):
    return 0 if state[0] > 19 else 1
```

In [5]:
```
print(policy(state))
```

```
1
```

Generando un episodio

A continuación, generamos un episodio usando la política dada, por lo que definimos una función llamada `generate_episode`, que toma la política como entrada y genera el episodio usando la política dada.

In [6]:
```
num_timestep = 100
```

In [7]:
```
def generate_episode(policy):

    #let's define a list called episode for storing the episode
    episode = []

    #initialize the state by resetting the environment
    state = env.reset()

    #then for each time step
    for i in range(num_timestep):
```

```
        #select the action according to the given policy
        action = policy(state)

        #perform the action and store the next state information
        next_state, reward, done, info = env.step(action)

        #store the state, action, reward into our episode list
        episode.append((state, action, reward))

        #If the next state is a final state then break the loop else update the
        if done:
            break

        state = next_state

    return episode
```

In [8]:
```
generate_episode(policy)
```

Out[8]:
```
[((13, 1, True), 1, 0.0), ((19, 1, True), 1, 0.0), ((14, 1, False), 1, -1.0)]
```

La salida tiene la forma de `[(estado, acción, recompensa)]` y como se dijo anteriormente, `estado` es una tupla con 3 elementos. Entonces, por ejemplo, en el estado `((13, 1, True), 1, 0.0)`, realizamos la acción `1` (pedir una carta) y recibimos una recompensa de `0`. La recompensa se asigna de la siguiente manera: `+1.0` si ganamos el juego, `-1.0` si perdemos el juego, `0` si el juego es un empate.

Calcular la función de valor

Aprendimos que para predecir la función de valor, generamos varios episodios utilizando la política dada y calculamos el valor del estado como un rendimiento promedio en varios episodios. Veamos cómo implementar eso.

In [9]:
```
total_return = defaultdict(float)
N = defaultdict(int)
```

In [10]:
```
num_iterations = 500000
```

In [11]:
```
#then, for every iteration
for i in range(num_iterations):

    #generate the episode using the given policy, that is, generate an episode u
    #function we defined earlier
    episode = generate_episode(policy)

    #store all the states, actions, rewards obtained from the episode
    states, actions, rewards = zip(*episode)

    #then for each step in the episode
    for t, state in enumerate(states):

            #compute the return R of the state as the sum of reward
```

```
            R = (sum(rewards[t:]))

            #update the total_return of the state
            total_return[state] =  total_return[state] + R

            #update the number of times the state is visited in the episode
            N[state] =  N[state] + 1
```

In [12]:
```
total_return = pd.DataFrame(total_return.items(),columns=['state', 'total_return
```

In [13]:
```
N = pd.DataFrame(N.items(),columns=['state', 'N'])
```

In [14]:
```
df = pd.merge(total_return, N, on="state")
```

In [15]:
```
df.head(10)
```

Out[15]:

	state	total_return	N
0	(18, 10, True)	-1348.0	3118
1	(16, 10, False)	-14209.0	20675
2	(14, 1, False)	-3320.0	4798
3	(14, 10, False)	-12164.0	19415
4	(8, 3, False)	-686.0	1211
5	(15, 3, False)	-3270.0	5069
6	(19, 9, False)	-4047.0	5634
7	(12, 4, False)	-2310.0	4435
8	(14, 4, False)	-2855.0	4835
9	(21, 4, False)	3430.0	3872

In [16]:
```
df['value'] = df['total_return']/df['N']
```

In [17]:
```
df.head(10)
```

Out[17]:

	state	total_return	N	value
0	(18, 10, True)	-1348.0	3118	-0.432328
1	(16, 10, False)	-14209.0	20675	-0.687255
2	(14, 1, False)	-3320.0	4798	-0.691955
3	(14, 10, False)	-12164.0	19415	-0.626526
4	(8, 3, False)	-686.0	1211	-0.566474
5	(15, 3, False)	-3270.0	5069	-0.645098

	state	total_return	N	value
6	(19, 9, False)	-4047.0	5634	-0.718317
7	(12, 4, False)	-2310.0	4435	-0.520857
8	(14, 4, False)	-2855.0	4835	-0.590486
9	(21, 4, False)	3430.0	3872	0.885847

Evaluemos el valor del estado `(21,9,Falso)`, como podemos observar, el valor de la suma de nuestras cartas ya es 21 por lo que este es un buen estado y debería tener un valor alto. Veamos cuál es nuestro valor estimado del estado:

In [18]:
```
df[df['state']==(21,9,False)]['value'].values
```

Out[18]:
```
array([0.94230769])
```

Ahora, verifiquemos el valor del estado `(5,8,Falso)`. Como podemos notar, el valor de la suma de nuestras cartas es apenas 5 e incluso la única carta del crupier tiene un valor alto, 8; en este caso, el valor del estado debería ser menor. Veamos cuál es nuestro valor estimado del estado:

In [19]:
```
df[df['state']==(5,8,False)]['value'].values
```

Out[19]:
```
array([-0.39777778])
```